GOT THIS ANSWER? YOU'RE IN THE RIGHT PLACE.

24

A Station A: What does this print?

Trace it on your recording sheet, then find the sheet with your answer at the top.

function.py

```
1 def function(n):
2     if n == 1:
3         return 1
4     else:
5         return 3 * function(n-1)
6
7 print(function(4))
```

pots

B

Station B: What does this print?

Trace it on your recording sheet, then find the sheet with your answer at the top.

function.py

```
1 def function(n):
2     if n == 1:
3         return 0
4     else:
5         return 1 + function(n // 2)
6
7 print(function(100))
```

22

C

Station C: What does this print?

Trace it on your recording sheet, then find the sheet with your answer at the top.

function.py

```
1 def function(x, y):
2     if x > y:
3         return x - y
4     else:
5         return function(x+3, y-1)
6
7 print(function(2, 12))
```

GOT THIS ANSWER? YOU'RE IN THE RIGHT PLACE.

3 2 1 2 3

(printed one per line)

D

Station D: What does this print?

Trace it on your recording sheet, then find the sheet with your answer at the top.

function.py

```
1 def function(n):
2     if n == 1:
3         return 2
4     else:
5         x = function(n-1)
6         print(x)
7         return x * 2
8
9 print(function(4))
```

27

E

Station E: What does this print?

Trace it on your recording sheet, then find the sheet with your answer at the top.

function.py

```
1 def function(s):
2     if len(s) == 0:
3         return ""
4     else:
5         return function(s[1:]) + s[0]
6
7 print(function("stop"))
```

GOT THIS ANSWER? YOU'RE IN THE RIGHT PLACE.

2 4 8 16

(printed one per line)

F

Station F: What does this print?

Trace it on your recording sheet, then find the sheet with your answer at the top.

function.py

```
1 def function(n):
2     if n < 10:
3         return n
4     else:
5         return n % 10 * function(n // 10)
6
7 print(function(324))
```

6

G

Station G: What does this print?

Trace it on your recording sheet, then find the sheet with your answer at the top.

function.py

```
1 def function(a):
2     if len(a) == 0:
3         return 0
4     elif a[0] > 5:
5         return a[0] + function(a[1:])
6     else:
7         return function(a[1:])
8
9 print(function([7, 2, 9, 5, 6]))
```

2

H Station H: What does this print?

Trace it on your recording sheet, then find the sheet with your answer at the top.

function.py

```
1 def function(n):
2     if n == 1:
3         print(1)
4     else:
5         print(n)
6         function(n-1)
7         print(n)
8
9 function(3)
```

Recursion Scavenger Hunt

Name: _____

Date: _____

Start at any station. Trace the code and write down what it prints. Then find the sheet with **your answer printed big at the top**. That's your next station. Keep going until you end up back where you started. If you can't find your answer anywhere, re-trace!

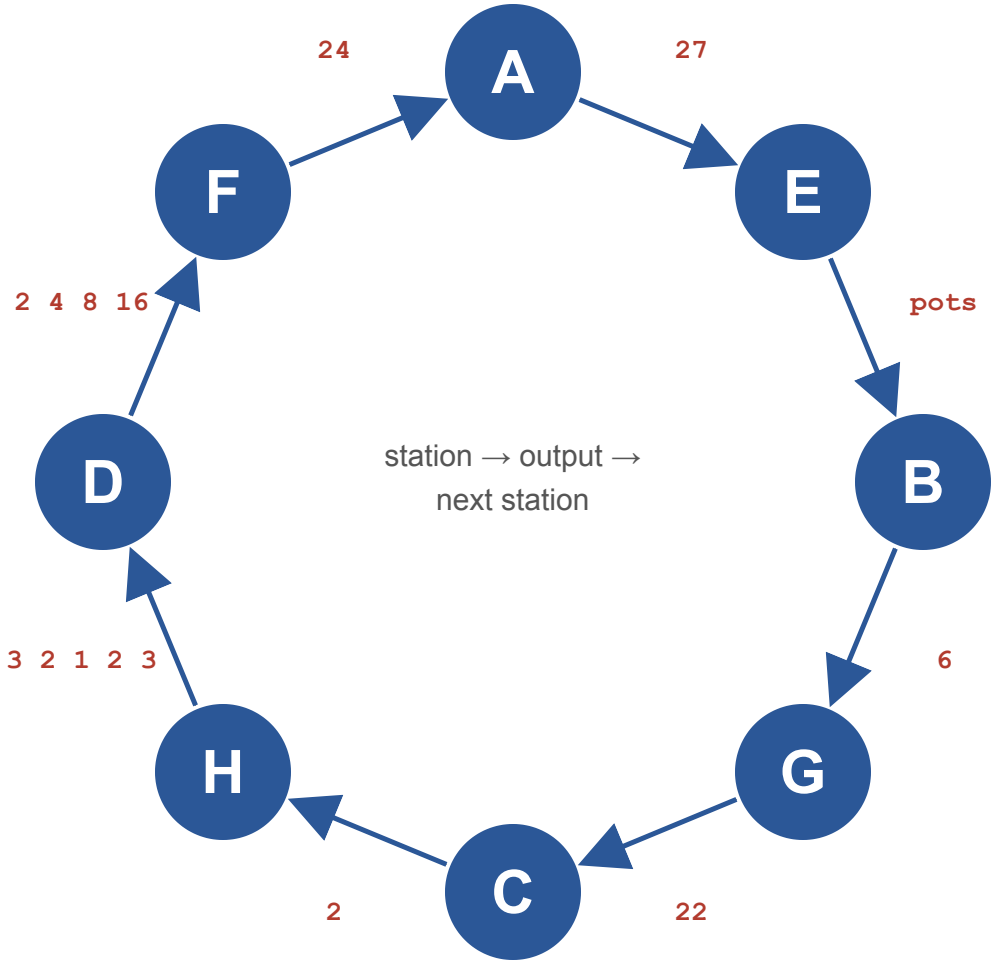
Start station: _____

	Station	Output	Base case reached when… / call stack notes
1			
2			
3			
4			
5			
6			
7			
8			

Teacher Key: Recursion Scavenger Hunt

Setup: print the 8 station sheets (A–H) one-sided and post them around the room in any order. Give each student or pair a recording sheet and a different start station. Each answer appears on exactly one sheet, so a wrong trace leads nowhere and tells students to re-check their work. Multi-line outputs are shown on one line on the sheets.

Full loop: A → E → B → G → C → H → D → F → A



Teacher Key: Answers by Station

Station	Call	Output	Go to
A	`print(function(4))`	**`27`**	**E**
E	`print(function("stop"))`	**`pots`**	**B**
B	`print(function(100))`	**`6`**	**G**
G	`print(function([7, 2, 9, 5, 6]))`	**`22`**	**C**
C	`print(function(2, 12))`	**`2`**	**H**
H	`function(3)`	**`3 2 1 2 3`**	**D**
D	`print(function(4))`	**`2 4 8 16`**	**F**
F	`print(function(324))`	**`24`**	**A**

Watch for these mistakes:

1. **"stop" instead of "pots"**: s[0] is added *after* the call returns, so the string comes back reversed.
2. **2 4 8 instead of 2 4 8 16**: the final print(function(4)) prints the returned 16 too.
3. **3 2 1 instead of 3 2 1 2 3**: the second print(n) in each frame runs as frames are popped.
4. **Halving 100**: 25 // 2 is 12, not 12.5. The calls go 100, 50, 25, 12, 6, 3, 1, which makes 6 steps.
5. **Counting base-case frames**: in step-count problems the base case adds 0, not 1.